# Homework 10

Solutions for Questions 1 and 2 will be available after Monday, November 2, 2026. No solution will be provided for Question 3.

Use the RLT package for Questions 1 and 2: `RLT` in R or `rlt` in Python. The node-size parameter is `nmin` in R and `min_samples_leaf` in Python; both stop splitting a node when it contains at most that many observations. The candidate-predictor parameter is `mtry` in both languages. Use ordinary random forests, with reinforcement turned off.

## Question 1: Bias-variance trade-off simulation

Two important tuning parameters of random forests are node size and `mtry`. We have performed some preliminary analysis in class. The goal of this question is to perform a larger simulation study for these two tuning parameters.

Consider a regression model with five jointly normally distributed predictors. Each predictor has mean 0 and variance 1, and the correlation between any two different predictors is 0.3. The response is

$$Y = X_1 + X_3 + X_5 + \varepsilon, \qquad \varepsilon \sim N(0,1),$$

where the noise is independent of the predictors. Thus the true regression function is $f(\mathbf{x}) = x_1 + x_3 + x_5$.

a. Set up a simulation study with 100 repetitions. Generate 200 test predictor vectors once and keep them fixed. In each repetition, generate 200 new training observations and use the same training data for all tuning settings. Fit 500-tree forests, drawing 200 observations with replacement for each tree. Make the following two comparisons:

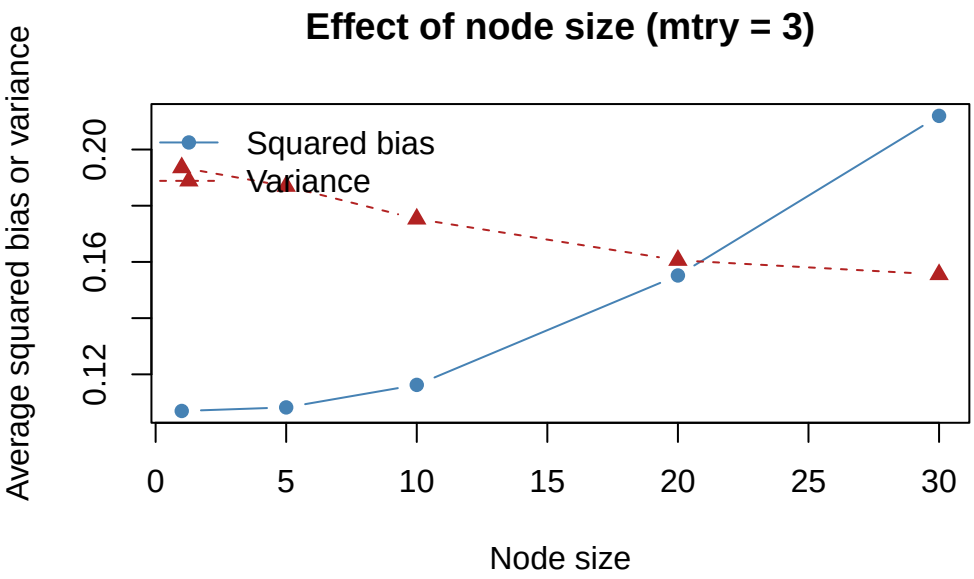
   - Fix `mtry` at 3 and vary node size over $1, 5, 10, 20, 30$.
   - Fix node size at 10 and vary `mtry` over $1, 2, 3, 4, 5$.

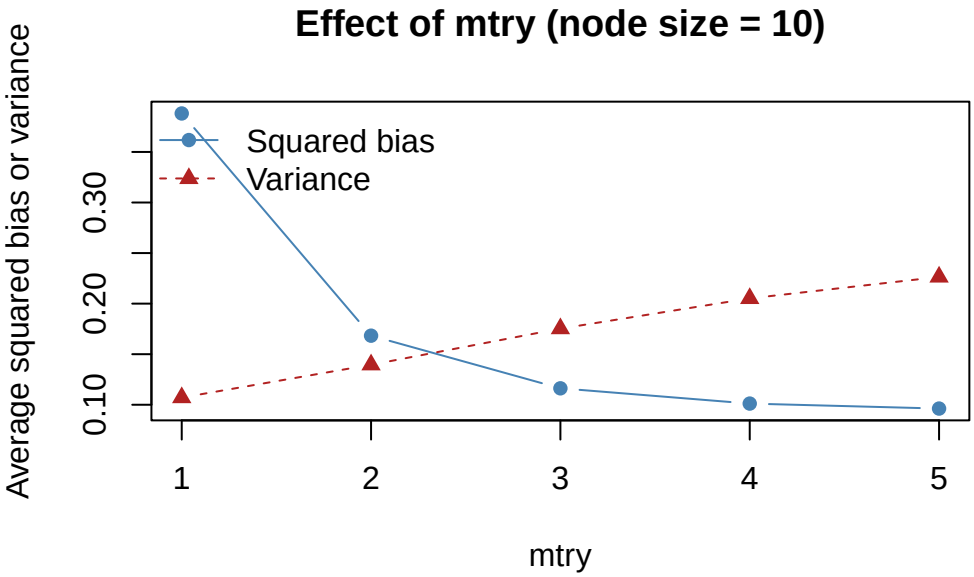
   Leave all other fitting parameters at their defaults. Record the predictions at each test point across repetitions. For each setting, estimate the squared bias and variance at each test point, then average them over the 200 test points. Compare predictions with $f(\mathbf{x})$ when estimating bias. Produce two figures, one for each tuning parameter, showing the averaged squared bias and variance. Set a random seed and report it.

| nodesize | mtry | Squared_bias | Variance |
|---|---|---|---|
| 1 | 3 | 0.1070 | 0.1936 |
| 5 | 3 | 0.1082 | 0.1869 |
| 10 | 3 | 0.1163 | 0.1753 |
| 20 | 3 | 0.1552 | 0.1606 |
| 30 | 3 | 0.2120 | 0.1555 |

| | nodesize | mtry | Squared_bias | Variance |
|---|---|---|---|---|
| 6 | 10 | 1 | 0.3881 | 0.1071 |
| 7 | 10 | 2 | 0.1683 | 0.1395 |

| | nodesize | mtry | Squared_bias | Variance |
|---|---|---|---|---|
| 3 | 10 | 3 | 0.1163 | 0.1753 |
| 9 | 10 | 4 | 0.1012 | 0.2052 |
| 10 | 10 | 5 | 0.0962 | 0.2264 |




I used random seed 432 and performed 100 repetitions. I generated 200 test predictor vectors once and kept them fixed. In each repetition, I generated 200 new training observations and used the same training data for every tuning setting. Each forest contained 500 trees, with each tree drawing a bootstrap sample of 200 observations.

At test point $x_i$, I estimated squared bias by

$$\widehat{\text{Bias}}^2(x_i) = \left[ \frac{1}{100} \sum_{b=1}^{100} \hat{f}_b(x_i) - f(x_i) \right]^2,$$

where $f(x_i) = x_{i1} + x_{i3} + x_{i5}$. I estimated prediction variance using the sample variance of the 100 predictions:

$$\widehat{\text{Var}}(x_i) = \frac{1}{99} \sum_{b=1}^{100} \left[ \hat{f}_b(x_i) - \bar{\hat{f}}(x_i) \right]^2.$$

I averaged each quantity over the 200 fixed test points. Bias was calculated relative to the true regression function, rather than noisy test responses. The figures show how these averaged estimates change with node size and `mtry`.

b. Discuss your results in terms of the bias-variance trade-off:

- How does a large node size, such as 30, affect the fit in this example?
- What are the benefits and drawbacks of using a large `mtry` value?
- If each tree instead sampled 100 observations with replacement, would you expect the preferred node size to shift toward smaller or larger values? Explain your reasoning. You may use an additional simulation to help, but a particular pattern is not guaranteed.

**Large node size.** A node size of 30 limits how far the trees can split, producing a smoother, less flexible fit. This can reduce variance but increase squared bias because the trees may miss changes in the true regression function. In this example, large terminal nodes can average together observations with noticeably different values of $x_1 + x_3 + x_5$, leading to underfitting.

**Large `mtry`.** A larger `mtry` gives each split more predictors to choose from, increasing the chance that an informative predictor is available. This can improve individual trees and reduce bias. However, trees may repeatedly select the same strong predictors, making their predictions more correlated and reducing the variance benefit of averaging. Therefore, a larger `mtry` does not necessarily produce the lowest overall prediction error.

**Smaller bootstrap samples.** If each tree sampled 100 observations instead of 200, I would expect the preferred node size to shift toward smaller values. With fewer observations per tree, the same node size represents a larger fraction of the sample and can stop splitting sooner. A smaller node size could preserve flexibility and reduce underfitting. However, smaller samples also make split estimates less stable, so allowing deeper trees could increase variance. The preferred setting depends on the balance between these effects, and a shift toward smaller values is not guaranteed.

## Question 2: Tuning random forests for MNIST

Use `mnist-first2000.csv.gz` from the combined ZIP. It contains 2,000 MNIST observations, with the digit label `Digit` and 784 pixel values.

- From the first 1,000 observations, keep digits 1, 6, and 7 as the training data. From the remaining 1,000 observations, keep the same digits as the test data.
- Fit PCA to the training pixels, centering but not scaling them. Use the first 20 principal components as predictors, and project the test observations using the same training means and loadings.

Complete the following tasks using RLT classification forests with 1,000 trees. For each tree, draw a bootstrap sample of the same size as the training set.

a. Fit a random forest with the default `mtry` and node-size settings. Report the out-of-bag (OOB) misclassification rate.

```
Package is not compiled with OpenMP (omp.h).
```

| Training_size | Test_size | Default_mtry | Default_nmin | OOB_error |
|---|---|---|---|---|
| 327 | 317 | 6 | 5 | 0.0306 |

I used digits 1, 6, and 7 from the first 1,000 observations for training and the same digits from the remaining 1,000 observations for testing. PCA was fitted to the training pixels, centering without scaling. I retained the first 20 components and projected the test observations using the training means and loadings.

Using seed 432, I fitted 1,000 trees with bootstrap samples equal in size to the training set. The installed package's default settings were `mtry` = 6 and `nmin` = 5. The OOB misclassification rate was 0.0306, or 3.06%. Each observation's OOB prediction used only trees that excluded it from their bootstrap samples.

b. Construct a grid of `mtry` and node-size values to tune the model. Select the setting using OOB misclassification rate. Report the OOB error for each setting and the selected values. Calculate and present the selected forest's OOB permutation variable importance for the 20 principal components. Comment on your results.

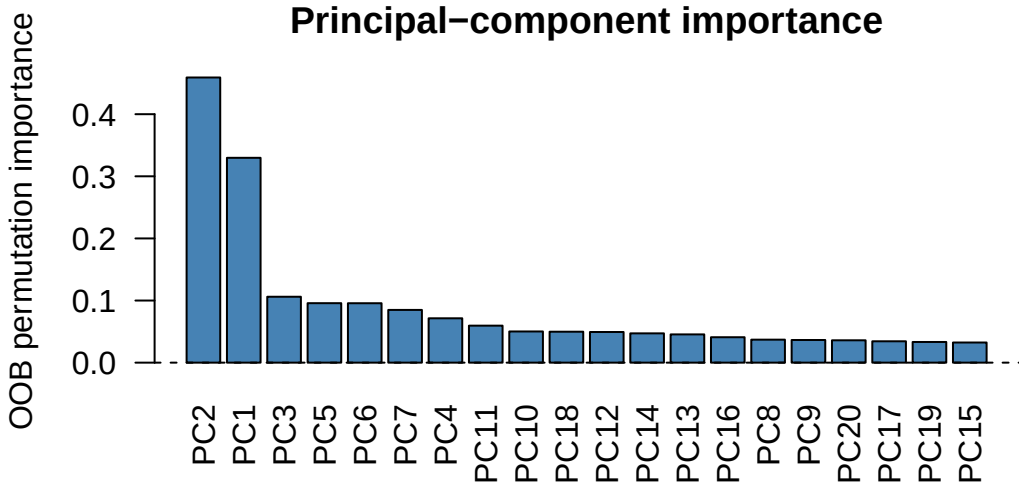
```
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
Package is not compiled with OpenMP (omp.h).
```

| mtry | nmin | OOB_error |
|---|---|---|
| 2 | 2 | 0.0183 |
| 6 | 2 | 0.0336 |
| 10 | 2 | 0.0336 |
| 20 | 2 | 0.0306 |
| 2 | 5 | 0.0275 |
| 6 | 5 | 0.0306 |
| 10 | 5 | 0.0336 |
| 20 | 5 | 0.0336 |
| 2 | 10 | 0.0245 |
| 6 | 10 | 0.0336 |
| 10 | 10 | 0.0306 |
| 20 | 10 | 0.0367 |
| 2 | 20 | 0.0245 |
| 6 | 20 | 0.0398 |
| 10 | 20 | 0.0428 |
| 20 | 20 | 0.0367 |
| 2 | 30 | 0.0367 |
| 6 | 30 | 0.0367 |

| mtry | nmin | OOB_error |
|---|---|---|
| 10 | 30 | 0.0550 |
| 20 | 30 | 0.0520 |

| mtry | nmin | OOB_error |
|---|---|---|
| 2 | 2 | 0.0183 |

| | Component | Importance |
|---|---|---|
| 2 | PC2 | 0.45908 |
| 1 | PC1 | 0.32978 |
| 3 | PC3 | 0.10610 |
| 5 | PC5 | 0.09572 |
| 6 | PC6 | 0.09565 |
| 7 | PC7 | 0.08490 |
| 4 | PC4 | 0.07140 |
| 11 | PC11 | 0.05956 |
| 10 | PC10 | 0.05021 |
| 18 | PC18 | 0.04982 |
| 12 | PC12 | 0.04933 |
| 14 | PC14 | 0.04715 |
| 13 | PC13 | 0.04548 |
| 16 | PC16 | 0.04093 |
| 8 | PC8 | 0.03700 |
| 9 | PC9 | 0.03644 |
| 20 | PC20 | 0.03597 |
| 17 | PC17 | 0.03440 |
| 19 | PC19 | 0.03327 |
| 15 | PC15 | 0.03242 |




I compared the settings shown in the table using OOB misclassification rate. The grid included the default settings, and all forests used 1,000 trees with full-size bootstrap samples. Exact ties were resolved by the original grid order.

The selected forest used `mtry` = 2 and `nmin` = 2, with OOB misclassification rate 0.0183. Its OOB error differed from the default forest's error by -1.22 percentage points.

The highest-ranked components were PC2, PC1, PC3. Permutation importance measures how much disrupting a component changes OOB prediction error. Larger positive values indicate greater predictive importance for this forest. Small or negative values can occur because of sampling variation. Components with the greatest pixel variance are not necessarily the most useful for distinguishing digits.

The minimum OOB error may be optimistic because it was used for model selection. Also, the OOB assessment treats the training-fitted PCA representation as fixed. The reserved test set provides a separate evaluation of the complete fitted pipeline.

c. Apply the selected model to the test data and report the misclassification rate and confusion matrix. Use the test labels only for this final assessment.

| | 1 | 6 | 7 |
|---|---|---|---|
| 1 | 102 | 0 | 2 |
| 6 | 0 | 106 | 0 |
| 7 | 1 | 1 | 105 |

| Errors | Test_size | Misclassification_rate |
|---|---|---|
| 4 | 317 | 0.0126 |

The selected forest misclassified 4 of 317 test observations. Its test misclassification rate was 0.0126, or 1.26%.

The confusion matrix has observed digits in rows and predicted digits in columns. The model settings were selected using OOB error, and the test labels were used only for the final assessment.

## Question 3: Planning a random-forest analysis skill

In earlier homework, some students wrote long skill files with detailed instructions, while others wrote short skills emphasizing their specific requirements or references. These approaches may perform differently depending on the AI and the task. A capable AI may already know standard ways to perform an analysis, so some instructions may be redundant, add little value, or sometimes make the analysis worse. It may be more useful to specify your personal choices for the analysis. An AI with more limited knowledge may benefit from detailed instructions that remind it of the steps to perform.

a. Write one paragraph describing how you would write a skill that directs an AI to perform a random-forest analysis and produce a report for a future dataset. Discuss how you would balance standard analysis instructions with your personal preferences, considering the AI's capabilities.

I would keep the skill fairly short instead of listing every step of a random-forest analysis. If the AI already knows the basics, I would focus on things I want it to pay attention to, like keeping test data separate, explaining its tuning choices, and making the results easy to read. If it seemed less familiar with the methods, I would add more guidance about OOB error and variable importance. My personal preferences would be a small comparison table, clearly labeled plots, and a short explanation of the final model. I would also avoid mentioning a specific dataset so I could use the skill again for another assignment.

b. Provide **10 instructions** you would consider including in the skill, **ordered from the most basic instructions to the more personal preferences**. Your list may cover computation, tuning, tables, figures, interpretation, presentation, or other aspects of a random-forest report.

You only need to discuss how the skill should be written. You are not required to write a complete skill file or test it.

1. Identify the response and predictors, and check whether the problem is classification or regression.

2. Check for missing values, incorrect variable types, and uneven class sizes.
3. Set a random seed and include enough code to reproduce the results.
4. Keep test data out of tuning and fit any preprocessing using training data only.
5. Fit a default random forest first so there is a baseline for comparison.
6. Try a reasonable range of `mtry` and node-size values, using the same evaluation method for each setting.
7. Choose the model using training-based results, then evaluate it on the separate test set.
8. Put the tuning settings and errors in a small table, with the selected setting clearly marked.
9. Use simple plots with readable labels to show tuning results and permutation importance.
10. Keep the report short and explain the main result, an important limitation, and what I would try next.